

Step 3 — A Triage Agent That Extracts a Structured Ticket From Free Text — Hints

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order. Each has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (the shape of the real code).

- [Hint 1 — The ticket, and the agent that fills it](#)
- [Hint 2 — The plan, and every failure](#)

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The ticket, and the agent that fills it

Basic Version

Write the ticket as a Pydantic model in `src/supportdesk/schemas.py` — the 4 fields from the README's Real Example. Two details matter:

- `customer_name: str | None` — `None` must be a valid answer, or the model will invent a name.
- `Literal[...]` for category and urgency, so only your exact words can come back.

Then `src/supportdesk/agents/triage.py` (plus an empty `agents/__init__.py`). The agent sends 2 messages — its system prompt and the complaint — with **no** chat history. It uses Doc04's `client.beta.chat.completions.parse(model=..., messages=..., response_format=SupportTicket)` and reads `completion.choices[0].message.parsed`.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
def triage_agent(client: OpenAI, message: str) -> SupportTicket | None:
    ...
```

It returns either a ticket or `None` — never anything half-valid:

- Wrap the `.parse(...)` call in `try / except ValidationError` → log a warning, return `None` .
- Check `reply.refusal` **before** `reply.parsed` (Doc04) → log it, return `None` .
- Otherwise return `reply.parsed` .

Put the system prompt in `prompts/triage.txt` and load it with `load_prompt("triage")` . Make it strict: "leave customer_name null unless the customer states their name — never guess one". The `None` in the schema only *allows* an empty name; the prompt is what *asks* for it.

Get the logger inside the function (`logger = get_logger(__name__)`), not at the top of the file. Files are imported before `load_config()` runs, so a top-level logger would miss `LOG_LEVEL` from `.env` .

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and every failure

Basic Version

In the loop, before the chat branch:

```
if message.startswith("/extract "):
    text = message.removeprefix("/extract ")
    ticket = triage_agent(client, text)
    if ticket is None: print a "couldn't turn that into a ticket" line
    else: print_ticket(ticket)
else:
    (Step 2's chat: trim, then stream)
```

`print_ticket()` prints the 4 fields on their own lines, with `(not given)` when the name is `None`.

Put the whole `if/else` inside a `try:`. The `except` blocks go right after it (Hint 2, Intermediate).

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

One `except` per failure, each with the reaction from Doc04's error table:

Catch	Why it happens	What to do
<code>openai.AuthenticationError</code>	key revoked mid-session	log error, print, <code>sys.exit(1)</code>
<code>openai.RateLimitError</code>	too many requests	log a warning, ask the user to wait
<code>openai.BadRequestError</code>	message too long	if <code>history[-1]["role"] == "user"</code> : <code>history.pop()</code> , then tell the user

Why `history.pop() : stream_message()` appended the user turn *before* the call failed. Left there, every later turn would resend it and fail again.

Then the files that make the step checkable:

- `data/sample/messages.json` — `{"trriage": [ ...the 3 inputs... ]}`.
- `scripts/triage_samples.py` — read the file and `json.loads()` its text (Doc02), run `trriage_agent()` on each message, print each ticket.
- `tests/unit/test_schemas.py` — a ticket with `customer_name=None` is accepted; `urgency="urgent"` raises `ValidationError`.
- `tests/integration/test_live_agents.py` — memory, the bad-key check, the 3 tickets, and "input 2 has no name".

Full solution: [Show me the solution](#)